

## **ReplacementManager.cpp**

The fixed-size heap is allocated with `new[]`. I track free/used regions as `Block` nodes in a `std::list` because the blocks could then be easily split and merged when memory is allocated or freed. Each block has an offset, size, an `isFree` flag, and an optional owner for handle-owned blocks. I keep the list sorted by offset and non-overlapping. I also coalesce adjacent free blocks in `coalesce_neighbors()` so the free space doesn't get fragmented into tiny pieces. The allocation mode First-fit (`alloc_first_fit`) finds the first free block that fits and split if needed. And Best-fit (`alloc_best_fit`) finds the smallest free block that fits and also splits if needed. I free with `dealloc(p)`. I check that the pointer belongs to the heap, that it points to the start of a block (not the interior), and that it's not already free. I mark it free and coalesce. I print the heap with `print_map()` for debugging. For handles, I keep a `std::list<HandleEntry>` with a master pointer (the thing the user's handle points to), a locked flag, and an alive flag. Defrag (`handle_defrag`) packs unlocked handle-owned blocks left using `memmove` and updates their master pointers, then rebuilds the block list with proper free gaps. I print the heap with `print_map()` for debugging. For handles (movable allocations), I keep a `std::list<HandleEntry>` with a master pointer, a locked flag, and an alive flag. Defrag (`handle_defrag`) packs unlocked handle-owned blocks left using `memmove` and updates their master pointers, then rebuilds the block list with proper free gaps.

## **replacementManager.h**

I define the `Block` and `HandleEntry` structs and expose the manager API. I list the globals (`gHeapBase`, `gHeapSize`, `gBlocks`, `gHandles`) and all functions: `init/shutdown`, `first-fit/best-fit`, `free`, `map printing`, and the handle helpers (`alloc bytes for a handle`, `free`, `lock/unlock`, `defrag`, `lookup`). I Used a namespace (`ReplacementManager`) to avoid naming conflicts with the C interface.

## **replacementMalloc.cpp**

`mm_init()` / `mm_shutdown()` wrap the manager. I use `ensure_init()` so the heap is created on first use. `myMalloc(size_t n)` picks `FIRST_FIT` or `BEST_FIT` based on `ALGORITHM_CHOICE`. When verbose is on, I print the allocation and the heap map. `myFree(void* p)` frees a raw pointer (no move ever happens for raw pointers). `printHeapMap()` just calls the manager's map printer. For handles, I added: `Handle makeHandle(nElems, elemSize)`: allocates `nElems * elemSize` bytes via the manager and returns a handle (a pointer to the handle entry's master pointer). `hfree(Handle h)`: frees the handle's block and marks the entry dead. `lock(h)` / `unlock(h)`: pins/unpins the block for defrag. `defrag()`: compacts unlocked handle blocks and updates the master pointers. I used a handle table so the allocator can move blocks safely. Raw `myMalloc` pointers never move, so old C code still works.

## **replacementMalloc.h**

C API: `mm_init`, `mm_shutdown`, `myMalloc`, `myFree`, `printHeapMap`, the handle type and functions (`Handle`, `makeHandle`, `hfree`, `lock`, `unlock`, `defrag`). These C programs call the allocator without C++ details leaking out. I Used `typedef void* Handle` so it can hold any pointer type safely.

**script06.sh**

converts Windows CRLF line endings to Unix LF for all .txt files from the input folder. It takes three args: input dir, a scrap dir for temporary files, and an output dir.

**Limitations:**

The heapController.cpp and .h files are present since they are part of the project layout, but they are empty because